Machine learning Lab Observation

ITA0614 - Machine Learning

Experiment 5

Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python.

Name : Variganji Sumanth Babu

Reg.No : 192524447

Problem 1: Fruit Classification

Aim

To implement K-Nearest Neighbours using pure Python and classify a fruit from its weight and sweetness.

Algorithm

1. Store the labelled training samples and choose k = 3.
2. Calculate the normalized Euclidean distance from the new fruit to every sample.
3. Sort the distances and select the k nearest samples.
4. Count the labels among the neighbours and choose the majority class.
5. Display the nearest neighbours and the predicted fruit class.

Euclidean distance: $d(x, y) = \sqrt{\sum((x_i - y_i)^2)}$; the class is chosen by majority vote among the k nearest samples.

Program:

knn_a.py - Python 3

```python
# KNN with pure Python: fruit classification
from math import sqrt
from collections import Counter

# Each row is (weight in grams, sweetness, fruit)
data = [
    (150, 6, "Apple"), (170, 7, "Apple"), (140, 5, "Apple"),
    (130, 8, "Orange"), (160, 9, "Orange"), (120, 7, "Orange"),
    (110, 9, "Banana"), (100, 8, "Banana"), (125, 10, "Banana"),
]

def distance(a, b):
    # Scale features so weight does not dominate sweetness.
    return sqrt(((a[0] - b[0]) / 100) ** 2
                + ((a[1] - b[1]) / 10) ** 2)

def predict(point, k=3):
    ranked = sorted((distance(point, r[:2]), r[2]) for r in data)
    neighbours = ranked[:k]
    label = Counter(name for _, name in neighbours).most_common(1)[0][0]
    return label, neighbours

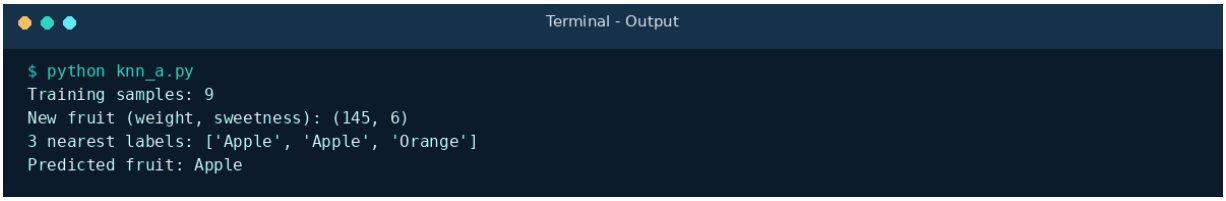
sample = (145, 6)
prediction, nearest = predict(sample, k=3)
print("Training samples:", len(data))
print("New fruit (weight, sweetness):", sample)
print("3 nearest labels:", [label for _, label in nearest])
print("Predicted fruit:", prediction)
```

Input

Fruit data set: 9 illustrative samples, with numeric features Weight (g) and Sweetness (score out of 10), and target Fruit (Apple, Orange or Banana). The examples are stored as a Python list.

Weight (g)	Sweetness	Fruit
150	6	Apple
170	7	Apple
140	5	Apple
130	8	Orange
160	9	Orange
120	7	Orange
110	9	Banana
100	8	Banana
125	10	Banana

Output



Result

The KNN classifier selects the majority fruit label among the three nearest samples. For the input weight of 145 g and sweetness score 6, the predicted class is Apple. The examples are illustrative.

Problem 2: Student Pass Prediction

Aim

To classify a student as Pass or Fail using KNN after reading training rows from CSV-formatted data.

Algorithm

1. Read the CSV records and separate the feature values from the result label.
2. Choose k = 3 and calculate Euclidean distances to the new student.
3. Sort the records by distance and take the three nearest neighbours.
4. Use majority voting to determine Pass or Fail.
5. Print the nearest-neighbour labels and final prediction.

Euclidean distance: $d(x, y) = sqrt(sum((x_i - y_i)^2))$; the class is chosen by majority vote among the k nearest samples.

Program:

knn_b.py - Python 3

```python
# KNN with CSV data: student pass prediction
import csv
from io import StringIO
from math import sqrt
from collections import Counter

csv_text = """StudyHours,AssignmentScore,Result
2,3,Fail
3,4,Fail
4,4,Fail
4,5,Fail
5,6,Pass
6,7,Pass
7,8,Pass
8,9,Pass
"""
rows = list(csv.DictReader(StringIO(csv_text.strip())))
data = [(float(r["StudyHours"]),
         float(r["AssignmentScore"]), r["Result"]) for r in rows]

def predict(sample, k=3):
    ranked = sorted((sqrt((x - sample[0]) ** 2 + (y - sample[1]) ** 2), label)
                    for x, y, label in data)
    nearest = ranked[:k]
    result = Counter(label for _, label in nearest).most_common(1)[0][0]
    return result, nearest

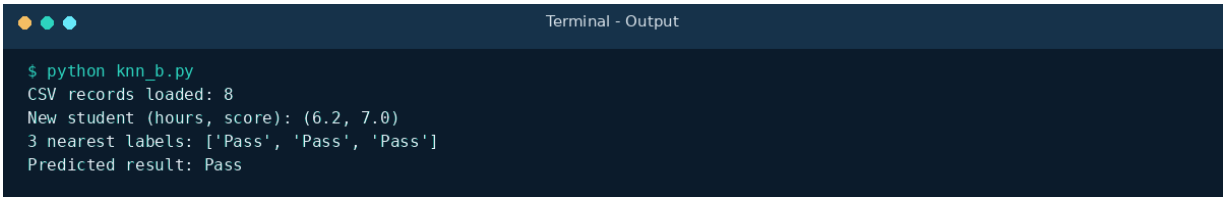
sample = (6.2, 7.0)
result, nearest = predict(sample, k=3)
print("CSV records loaded:", len(data))
print("New student (hours, score):", sample)
print("3 nearest labels:", [label for _, label in nearest])
print("Predicted result:", result)
```

Input

Student data set: 8 illustrative samples, with Study Hours and Assignment Score (0-10), and target Result (Pass / Fail). The CSV-formatted records are parsed with Python csv.DictReader.

Study Hours	Assignment Score	Result
2	3	Fail
3	4	Fail
4	4	Fail
4	5	Fail
5	6	Pass
6	7	Pass
7	8	Pass
8	9	Pass

Output



Result

The CSV-based KNN program classifies a student using study hours and assignment score. The sample is predicted as Pass from the majority labels of its three nearest training records.

Problem 3: Iris Flower Classification

Aim

To implement KNN with NumPy arrays and predict the species of an iris flower from four measurements.

Algorithm

1. Store the four flower measurements in X and species labels in y.
2. Calculate the Euclidean distance from the new flower to every row in X.
3. Select the k = 3 rows with the smallest distances.
4. Apply majority voting to the selected species labels.
5. Display the nearest labels and the predicted species.

Euclidean distance: $d(x, y) = sqrt(sum((x_i - y_i)^2))$; the class is chosen by majority vote among the k nearest samples.

Program:

knn_c.py - Python 3

```python
# KNN with NumPy: iris flower classification
import numpy as np
from collections import Counter

X = np.array([
    [5.1, 3.5, 1.4, 0.2], [4.9, 3.0, 1.4, 0.2],
    [4.7, 3.2, 1.3, 0.2], [7.0, 3.2, 4.7, 1.4],
    [6.4, 3.2, 4.5, 1.5], [6.9, 3.1, 4.9, 1.5],
    [6.3, 3.3, 6.0, 2.5], [5.8, 2.7, 5.1, 1.9],
    [7.1, 3.0, 5.9, 2.1],
])
y = np.array(["Setosa"] * 3 + ["Versicolor"] * 3
             + ["Virginica"] * 3)

def predict(sample, k=3):
    distances = np.sqrt(np.sum((X - sample) ** 2, axis=1))
    idx = np.argsort(distances)[:k]
    votes = Counter(y[idx])
    return votes.most_common(1)[0][0], y[idx].tolist()

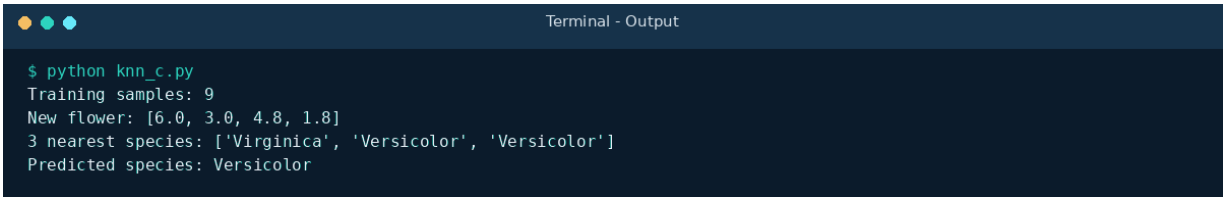
sample = np.array([6.0, 3.0, 4.8, 1.8])
result, neighbours = predict(sample, k=3)
print("Training samples:", len(X))
print("New flower:", sample.tolist())
print("3 nearest species:", neighbours)
print("Predicted species:", result)
```

Input

Iris data set: 9 representative samples from three species, using Sepal Length, Sepal Width, Petal Length and Petal Width (cm). Target class is Species (Setosa, Versicolor or Virginica); rows are represented with NumPy arrays.

Sepal L.	Sepal W.	Petal L.	Petal W.	Species
5.1	3.5	1.4	0.2	Setosa
4.9	3.0	1.4	0.2	Setosa
4.7	3.2	1.3	0.2	Setosa
7.0	3.2	4.7	1.4	Versicolor
6.4	3.2	4.5	1.5	Versicolor
6.9	3.1	4.9	1.5	Versicolor
6.3	3.3	6.0	2.5	Virginica
5.8	2.7	5.1	1.9	Virginica
7.1	3.0	5.9	2.1	Virginica

Output



Result

The NumPy implementation computes distances in the four-dimensional measurement space and predicts the most common species among the nearest three flowers. The supplied sample is classified as Versicolor.

Problem 4: Bank Loan Approval

Aim

To use a pandas DataFrame and a scaled KNeighborsClassifier model to classify a sample loan application.

Algorithm

1. Load the numeric loan records into a pandas DataFrame.
2. Separate the predictor columns from the Approved target.
3. Standardize the features so their different units do not dominate distance.
4. Fit a KNeighborsClassifier with k = 3 and predict the new application.
5. Display the prediction; the toy records do not define a real lending policy.

Euclidean distance: $d(x, y) = \sqrt{\sum((x_i - y_i)^2)}$; the class is chosen by majority vote among the k nearest samples.

Program:

knn_d.py - Python 3

```python
# KNN with pandas and scikit-learn: loan approval
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

rows = [
    [25,520,0.70,"No"], [30,560,0.65,"No"],
    [35,600,0.55,"No"], [45,620,0.45,"No"],
    [50,650,0.35,"Yes"], [55,680,0.30,"Yes"],
    [65,700,0.25,"Yes"], [75,730,0.20,"Yes"],
    [85,760,0.15,"Yes"], [95,780,0.10,"Yes"],
]
cols = ["IncomeK", "CreditScore", "DebtRatio", "Approved"]
df = pd.DataFrame(rows, columns=cols)
X = df[["IncomeK", "CreditScore", "DebtRatio"]]
y = df["Approved"]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_scaled, y)

sample = pd.DataFrame([[60, 690, 0.28]], columns=X.columns)
prediction = model.predict(scaler.transform(sample))[0]
print("Training applications:", len(df))
print("New application:", sample.iloc[0].to_dict())
print("Predicted approval:", prediction)
print("Note: illustrative data, not a real lending rule.")
```

Input

Bank Loan data set: 10 illustrative records with numeric features Income (thousands), Credit Score and Debt Ratio; target Approved (Yes / No). The pandas DataFrame is scaled before KNN is trained. This small data set is for demonstration only.

Income (k)	Credit Score	Debt Ratio	Approved
25	520	0.70	No
30	560	0.65	No
35	600	0.55	No
45	620	0.45	No
50	650	0.35	Yes
55	680	0.30	Yes
65	700	0.25	Yes
75	730	0.20	Yes
85	760	0.15	Yes
95	780	0.10	Yes

Output

Terminal - Output

```
$ python knn_d.py
Training applications: 10
New application: {'IncomeK': 60.0, 'CreditScore': 690.0, 'DebtRatio': 0.28}
Predicted approval: Yes
Note: illustrative data, not a real lending rule.
```

Result

The pandas and scikit-learn implementation standardizes the loan features before applying KNN. The sample application is classified using the small illustrative data set; this is not a real credit decision rule.

Problem 5: Mobile Price Range

Aim

To classify a mobile phone into a price range using KNN, feature scaling and scikit-learn.

Algorithm

1. Create the mobile phone feature matrix and price-range labels.
2. Apply MinMaxScaler to map features to a common range.
3. Train KNeighborsClassifier using k = 3 nearest neighbours.
4. Scale the new phone features using the same fitted scaler.
5. Predict and display the phone price category.

Euclidean distance: $d(x, y) = \sqrt{\sum((x_i - y_i)^2)}$; the class is chosen by majority vote among the k nearest samples.

Program:

knn_e.py - Python 3

```python
# KNN with scikit-learn: mobile price range
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier

rows = [
    [2,3000,5.5,"Low"], [3,3200,5.8,"Low"],
    [3,3500,6.0,"Low"], [4,3600,6.1,"Low"],
    [4,4000,6.2,"Mid"], [6,4200,6.4,"Mid"],
    [6,4500,6.5,"Mid"], [8,4700,6.6,"Mid"],
    [8,5000,6.7,"High"], [12,4800,6.8,"High"],
    [12,5000,6.9,"High"], [16,5200,7.0,"High"],
]
columns = ["RAM", "Battery", "Screen", "PriceRange"]
df = pd.DataFrame(rows, columns=columns)
X = df[["RAM", "Battery", "Screen"]]
y = df["PriceRange"]
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_scaled, y)

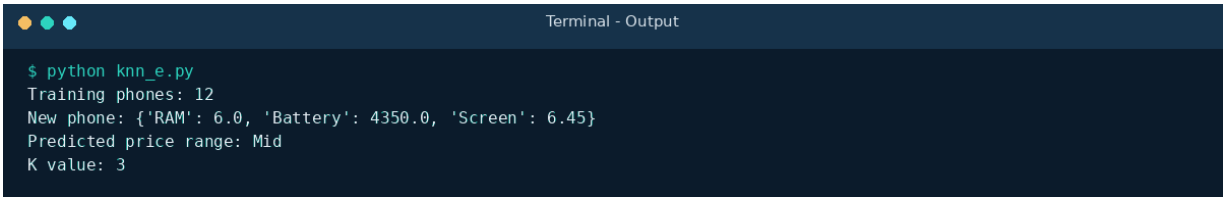
phone = pd.DataFrame([[6, 4350, 6.45]], columns=X.columns)
category = model.predict(scaler.transform(phone))[0]
print("Training phones:", len(df))
print("New phone:", phone.iloc[0].to_dict())
print("Predicted price range:", category)
print("K value:", model.n_neighbors)
```

Input

Mobile Price data set: 12 illustrative phones described by RAM (GB), Battery (mAh) and Screen Size (inches). The target Price Range is Low, Mid or High. Min-max scaling is used before distance-based classification.

RAM (GB)	Battery (mAh)	Screen (in)	Price Range
2	3000	5.5	Low
3	3200	5.8	Low
3	3500	6.0	Low
4	3600	6.1	Low
4	4000	6.2	Mid
6	4200	6.4	Mid
6	4500	6.5	Mid
8	4700	6.6	Mid
8	5000	6.7	High
12	4800	6.8	High
12	5000	6.9	High
16	5200	7.0	High

Output



Result

The scaled KNN model assigns the new phone to a price range based on the three nearest training examples. The output gives the predicted category for the sample phone; the data is for demonstration only.